Meal Prep & Meal Prep Chef

Weekly meal planner with a Telegram cooking assistant, built on n8n

Edgar Gusching • Computer Science Adventures (n8n) • October 2026

n8n (self-hosted) • Telegram • Spoonacular • Groq • Airtable • Supabase • Vercel

Target: Level 3 | **Finished and live**

https://meal-prep-intake-1.vercel.app • https://t.me/meal_prep_intake_bot

---

1. The need

People cooking on a budget lose hours every week picking recipes, checking the cost and writing a shopping list, then cook from a dozen browser tabs and a phone timer. Meal Prep splits the job in two: **you plan your week on the website**, and once that's done, **the Telegram bot *Meal Prep Chef* takes over in the kitchen**, telling you what's on tonight, walking you through each recipe step by step, timing your steps and answering any cooking question.

2. The trigger

One entry point per stage of the journey. **Plan:** the website posts its form to an n8n **Webhook**. **Cook:** an incoming **Telegram message** fires a Telegram Trigger on `@meal_prep_intake_bot`, the project's message trigger: commands such as `/today`, `/cook friday dinner`, `next`, `/timer 10`, or free text (*"how do I know the chicken is done?"*). **Testing on demand:** click *Plan my week* on the site (or `curl` the webhook), then message the bot from a phone or desktop; n8n's *Listen for test event* gives one debugged run.

3. Architecture: plan on the website, cook with the bot

Seven workflows serve one journey (Figure 1, page 2). The website is where you *plan*; the bot is where you *cook*, get help and get reminded. Airtable is the shared memory between the two.

Stage 1 — Plan your week on the website (workflows 1–5)

1. **Fill in the form** on the site, logged in with Supabase: budget, number of people, diet, equipment, what's already in the cupboard, and which lunches and dinners to cook.
2. **Generate the week** (workflow 1). The webhook answers at once with a plan reference, then keeps working: Spoonacular returns about thirty matching recipes, Groq picks the week within the budget while reusing ingredients, and Airtable stores every meal.
3. **See your week.** The site polls workflow 2 every few seconds and shows the plan as meal cards, with cost, time and nutrition.
4. **Adjust it.** Swap any meal you don't fancy (workflow 5), then validate the week to get one shopping list, merged by a Code node and sorted by aisle with prices by Groq (workflows 3–4).
5. **Hand over to the bot.** The *Cook with Telegram* button opens the bot with the plan reference (`/start MP-...`), which links the chat to the account.

Stage 2 — Cook with Meal Prep Chef on Telegram (workflows 6–7)

1. **Telegram Trigger → Read the message** (Code): intent and meal slot (day, lunch or dinner, in Paris time).
2. **Command Router** (Switch): `/start` and `/help` send the menu, `/start MP-...` saves the account link, `/timer N` waits and pings back; everything else goes on.

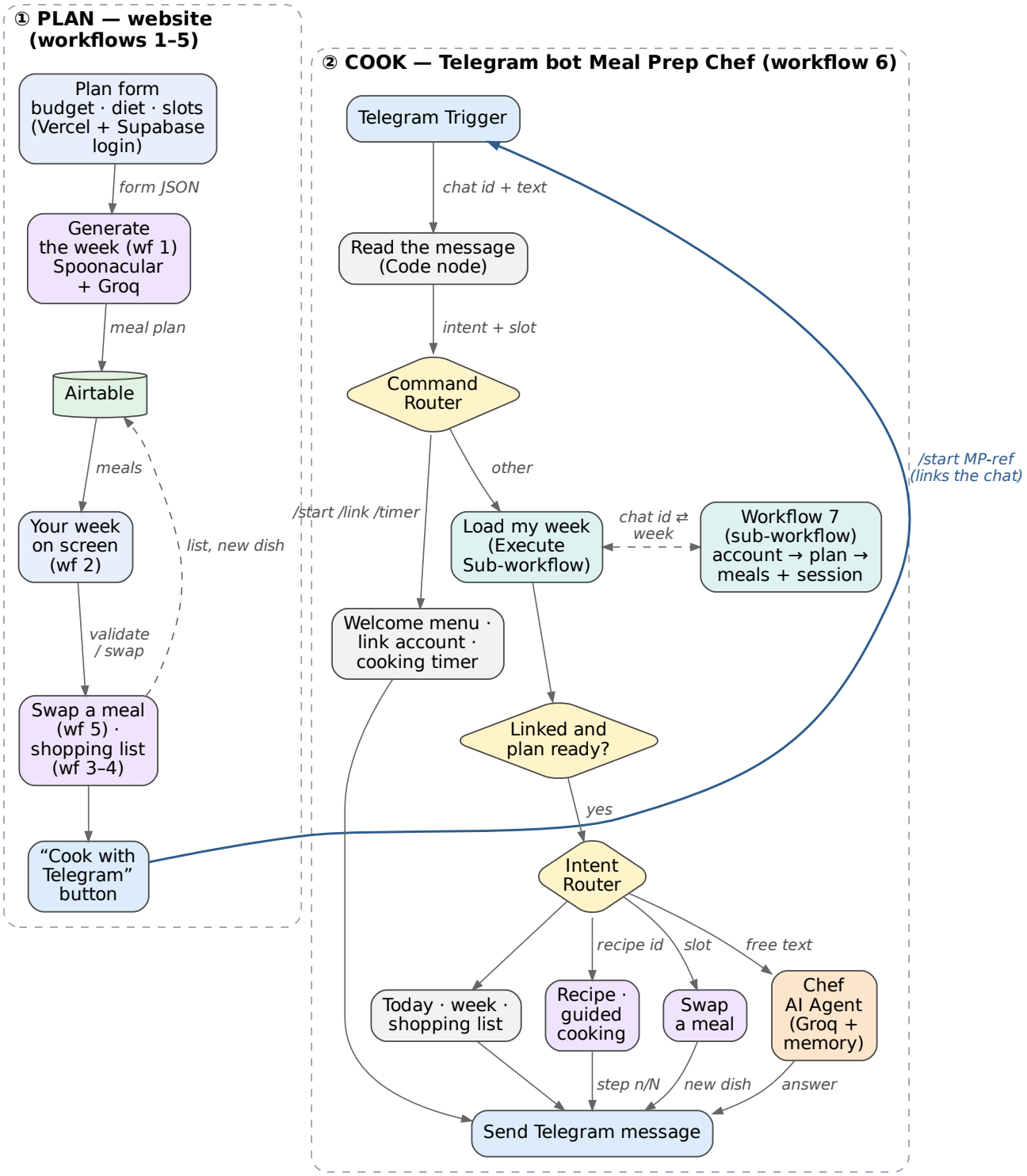


Figure 1. The user journey. ① Plan on the website (left), then the *Cook with Telegram* button hands over to ② the bot (right). Labels on the arrows show what travels between blocks. Blue: entry points and replies; yellow: routing; grey: Code nodes; purple: external APIs; orange: AI; green: database; teal: sub-workflow.

3. **Load my week** (Execute Sub-workflow → workflow 7): account, latest plan, its meals and the current cooking session, read from Airtable. If the chat isn't linked or no plan is ready, the bot explains what to do.
4. **Intent Router** (Switch) → *today, week, shopping list* straight from Airtable (no AI); *recipe and guided cooking* with Spoonacular's steps, `next`, `back`, `repeat` and step timers; *swap* through workflow 5; *free text* to the **Chef AI Agent** (Groq with memory), which knows the week and the step being cooked.
5. **Send Telegram message**: one shared node for every reply.

4. Level aimed for, and the cut line

Level 3, reached. Both the web planner and the Telegram bot are finished and running in production on a self-hosted n8n instance.

Cut line, in the order things would have been dropped if time ran short:

1. guided cooking sessions and timers → fall back to `/recipe`, which sends the whole recipe at once;
2. the agent's conversation memory → single-turn answers;
3. swapping a meal from Telegram → swaps only on the website.

Never cut: the weekly plan, Airtable persistence, and `/today`, which together carry the core value.

5. Minimum technical requirements

Requirement	Where it is in the project
If or Set node	If nodes *Linked and plan ready?*, *Recipe available?*, *Valid timer?*; a Set node attaches the intent to the week data before routing.
Code node	Message parser (intent and meal slot), flattening Spoonacular's steps, moving through a cooking session, merging ingredients for the shopping list.
Router or If/Switch	Two Switch nodes: *Command Router* (help, link, timer, rest) and *Intent Router* (eight routes).
API or MCP connection	Spoonacular (recipe pool and step-by-step instructions), Groq, and HTTP calls to the project's own webhooks (swap through workflow 5, status through workflow 2).
Message trigger	Telegram Trigger on `@meal_prep_intake_bot`.
Database	Airtable: requests, meals, shopping lists, Telegram links and cooking sessions. Supabase holds the user accounts.
AI node	*Chef* AI Agent with a Groq chat model and a window memory of the last ten messages. Groq is also used in the backend to compose the week, price the list and choose swaps.
Sub-workflow	Workflow 7, called with Execute Sub-workflow by every bot command: chat ID → account → latest plan → meals and cooking session.
Error handling and retries	Retry on fail (3 tries) on HTTP, Airtable writes and the Telegram sender; continue-on-error with readable fallback messages (no steps available, API quota hit, chat not linked); swap polling capped at nine tries; the backend records an *echec* status with its reason.